Publish a Component

Publish to PyPI

Publishing your Streamlit Component to PyPI makes it easily accessible to Python users around the world. This step is completely optional, so if you won't be releasing your component publicly, you can skip this section!

push_pin **Note**

For static Streamlit Components, publishing a Python package to PyPI follows the same steps as the core PyPI packaging instructions. A static Component likely contains only Python code, so once you have your setup.py file correct and generate your distribution files, you're ready to upload to PyPI.

Bi-directional Streamlit Components at minimum include both Python and JavaScript code, and as such, need a bit more preparation before they can be published on PyPI. The remainder of this page focuses on the bi-directional Component preparation process.

Prepare your Component

A bi-directional Streamlit Component varies slightly from a pure Python library in that it must contain pre-compiled frontend code. This is how base Streamlit works as well; when you `pip install streamlit`, you are getting a Python library where the HTML and frontend code contained within it have been compiled into static assets.

The component-template GitHub repo provides the folder structure necessary for PyPI publishing. But before you can publish, you'll need to do a bit of housekeeping:

1. Give your Component a name, if you haven't already
   - Rename the `template/my_component/` folder to `template/<component name>/`
   - Pass your component's name as the the first argument to `declare_component()`
2. Edit `MANIFEST.in`, change the path for recursive-include from `package/frontend/build *` to `<component name>/frontend/build *`

3. Edit `setup.py`, adding your component's name and other relevant info

4. Create a release build of your frontend code. This will add a new directory, `frontend/build/`, with your compiled frontend in it:

```
cd frontend
npm run build
```

5. Pass the build folder's path as the `path` parameter to `declare_component`. (If you're using the template Python file, you can set `_RELEASE = True` at the top of the file):

```
import streamlit.components.v1 as components

# Change this:
# component = components.declare_component("my_component", url="http://localhost:30

# To this:
parent_dir = os.path.dirname(os.path.abspath(__file__))
build_dir = os.path.join(parent_dir, "frontend/build")
component = components.declare_component("new_component_name", path=build_dir)
```

Build a Python wheel

Once you've changed the default `my_component` references, compiled the HTML and JavaScript code and set your new component name in `components.declare_component()`, you're ready to build a Python wheel:

1. Make sure you have the latest versions of setuptools, wheel, and twine

2. Create a wheel from the source code:

```
# Run this from your component's top-level directory; that is,
# the directory that contains `setup.py`
python setup.py sdist bdist_wheel
```

Upload your wheel to PyPI

With your wheel created, the final step is to upload to PyPI. The instructions here highlight how to upload to Test PyPI, so that you can learn the mechanics of the process without worrying about messing anything up. Uploading to PyPI follows the same basic procedure.

1. Create an account on Test PyPI if you don't already have one
   - Visit https://test.pypi.org/account/register/ and complete the steps
   - Visit https://test.pypi.org/manage/account/#api-tokens and create a new API token. Don't limit the token scope to a particular project, since you are creating a new project. Copy your token before closing the page, as you won't be able to retrieve it again.
2. Upload your wheel to Test PyPI. `twine` will prompt you for a username and password. For the username, use **__token__**. For the password, use your token value from the previous step, including the `pypi-` prefix:

```
python -m twine upload --repository testpypi dist/*
```

3. Install your newly-uploaded package in a new Python project to make sure it works:

```
python -m pip install --index-url https://test.pypi.org/simple/ --no-deps example-pkg
```

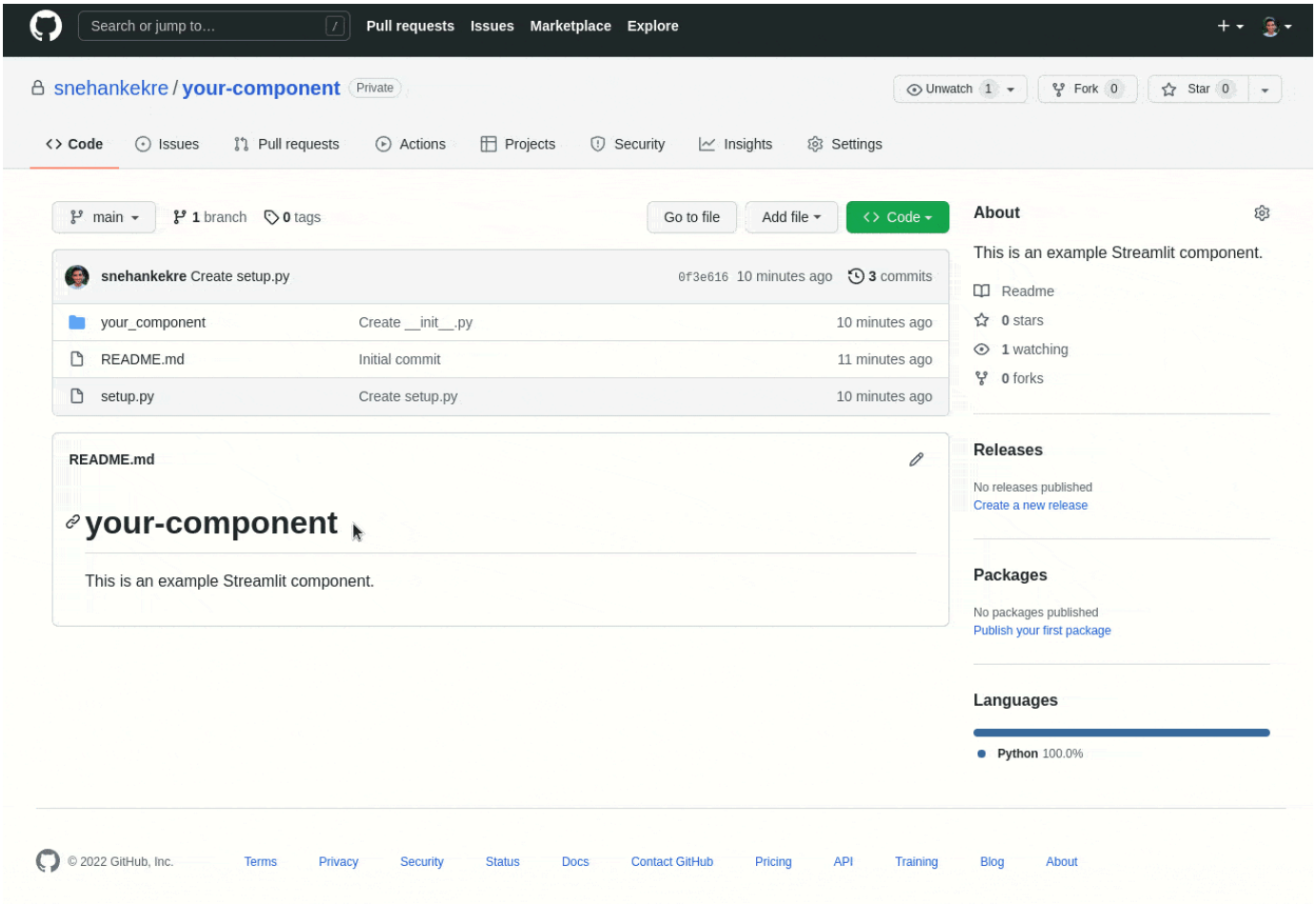
If all goes well, you're ready to upload your library to PyPI by following the instructions at https://packaging.python.org/tutorials/packaging-projects/#next-steps.

Congratulations, you've created a publicly-available Streamlit Component!

Promote your Component!

We'd love to help you share your Component with the Streamlit Community! To share it:

1. If you host your code on GitHub, add the tag `streamlit-component`, so that it's listed in the GitHub **streamlit-component** topic:



Add the streamlit-component tag to your GitHub repo

2. Post on the Streamlit Forum in Show the Community!. Use a post title similar to "New Component: `<your component name>`, a new way to do X".

3. Add your component to the Community Component Tracker.

4. Tweet us at @streamlit so that we can retweet your announcement for you.

Our Components Gallery is updated approximately every month. Follow the above recommendations to maximize the liklihood of your component landing in our Components Gallery. Community Components featured in our docs are hand-curated on a less-regular basis. Popular components with many stars and good documentation are more likely to be selected.